Contrato de Servicio: File Sync Service

1. Información general

Campo	Valor
Nombre del servicio	File Sync Service
Descripción	Servicio encargado de mantener sincronizado el contenido del Home (y directorios compartidos) entre los distintos dispositivos de un usuario y el Repository central, detectando cambios, versionando y resolviendo conflictos.
Objetivo	Garantizar que un usuario vea el mismo estado de sus archivos sin importar desde qué dispositivo trabaje, sin pérdida de datos ante ediciones concurrentes.
Responsabilidad	Sincronización automática, programada y al conectarse a la red; detección de cambios; versionado (delegado en Storage); resolución de conflictos; comparación de archivos.
Tecnología	Go + gRPC (impuesta)
Protocolo de comunicación	gRPC (contrato definido como servicio + mensajes, equivalente conceptual a un `.proto`)
Servicios consumidores	Clientes de escritorio (Windows, Linux) principalmente; Web de forma limitada (sincronización bajo demanda, no en segundo plano); móviles de forma opcional.
Servicios proveedores	Authentication Service (`ValidarToken`), Distributed Storage Service (`SubirArchivo`, `DescargarArchivo`, `ListarVersiones`, `RestaurarVersion`, `ObtenerMetadatos`).

2. Funcionalidades (operaciones expuestas, como métodos de servicio gRPC)

1. IniciarSincronizacion
2. DetectarCambiosLocales
3. SincronizarArchivo (subida delta)
4. DescargarCambiosRemotos (bajada delta)
5. CompararArchivos
6. ResolverConflicto

7. ProgramarSincronizacion
8. ConfigurarPoliticaSincronizacion
9. ObtenerEstadoSincronizacion
10. ObtenerHistorialSincronizacion
11. CancelarSincronizacion
12. RegistrarDispositivo

---

3. Contrato de cada operación

3.1 IniciarSincronizacion

Nombre: `IniciarSincronizacion` **Descripción:** Arranca una sesión de sincronización entre un dispositivo cliente y el Repository, ya sea por conexión a red, por disparo manual o por programación. **Protocolo:** gRPC (streaming bidireccional recomendado, dado que puede haber múltiples archivos en tránsito)

Parámetros de entrada (mensaje `SolicitudSincronizacion`):

Nombre	Tipo	Obligatorio	Descripción	Restricciones/Validaciones
`token`	string	Sí	Token de sesión válido	Validado contra Authentication Service
`dispositivoId`	string	Sí	Identificador único y persistente del dispositivo cliente	Registrado previamente vía `RegistrarDispositivo`
`rutaBase`	string	Sí	Directorio raíz a sincronizar (Home o subdirectorio)	Debe existir y el usuario debe tener permiso
`modoDisparo`	enum	Sí	`MANUAL`, `AUTOMATICO_AL_CONECTAR`, `PROGRAMADO`	—
`marcaTiempoUltimaSync`	datetime	No	Última sincronización exitosa conocida por el cliente	Si se omite, se asume sincronización completa (primer uso)

Datos de salida (mensaje `SesionSincronizacion`, stream):

Nombre	Tipo	Descripción
`sesionId`	UUID	Identificador de la sesión de sincronización
`cambiosDetectados`	int	Total de archivos con diferencias a resolver
`estado`	enum	`EN_PROGRESO`, `COMPLETADA`, `CON_CONFLICTOS`, `FALLIDA`
`progreso`	float (0-1)	Avance de la sincronización, emitido periódicamente por el stream

Posibles errores:

Código	Descripción	Posible solución
`SYNC-001`	Dispositivo no registrado	Ejecutar `RegistrarDispositivo` primero
`SYNC-002`	Ruta base no accesible o sin permiso	Verificar permisos vía Shared File Service
`SYNC-003`	Otra sesión de sincronización ya en curso para el mismo dispositivo y ruta	Esperar a que termine o cancelarla explícitamente
`SYNC-004`	Storage no disponible	Reintentar con backoff; alertar a Monitoring

Reglas de negocio:

- Solo puede existir **una sesión activa por combinación dispositivo+rutaBase**; sesiones concurrentes sobre la misma ruta se rechazan explícitamente en lugar de producir resultados indeterminados.
- El modo `AUTOMATICO_AL_CONECTAR` se dispara internamente cuando el cliente detecta reconexión de red — la lógica de "detección de reconexión" vive en el cliente, no en este servicio; el servicio solo recibe la solicitud resultante.

Seguridad: Requiere token válido y permiso de lectura/escritura sobre `rutaBase` (verificado en conjunto con Shared File Service / Storage).

Dependencias: Authentication Service, Distributed Storage Service.

Flujo interno:

1. Presentación (gRPC) recibe la solicitud y abre el stream de respuesta.
2. Aplicación valida token y permisos.

3. Dominio (capa de Detección de Cambios/Reconciliación) compara el estado local reportado por el cliente contra los metadatos actuales en Storage (vía `ObtenerMetadatos`/`ListarVersiones`).
4. Se clasifican los archivos en: sin cambios, solo cambiaron localmente, solo cambiaron remotamente, cambiaron en ambos lados (conflicto potencial).
5. Para los que no tienen conflicto, se ejecutan automáticamente `SincronizarArchivo` o `DescargarCambiosRemotos` según corresponda.
6. Para los que sí tienen conflicto, se emiten al stream como pendientes de `ResolverConflicto`.
7. Se actualiza el estado de la sesión y se emite evento `SincronizacionCompletada` o `SincronizacionConConflictos`.

3.2 DetectarCambiosLocales

Descripción: El cliente reporta el estado de sus archivos locales (rutas, tamaños, checksums, marca de tiempo de modificación) para que el servicio calcule el diff contra el estado remoto. **Protocolo:** gRPC (mensaje unario o streaming si son muchos archivos)

Entrada: `token`, `dispositivoId`, `listaArchivosLocales` (array de `{ruta, checksum, tamano, fechaModificacion}`).

Salida: `listaCambios` (array de `CambioDetectado` con su clasificación: `NUEVO_LOCAL`, `NUEVO_REMOTO`, `MODIFICADO_LOCAL`, `MODIFICADO_REMOTO`, `CONFLICTO`, `ELIMINADO_LOCAL`, `ELIMINADO_REMOTO`).

Errores: `SYNC-005` (formato de checksum inválido), `SYNC-002` (permiso insuficiente sobre alguna de las rutas reportadas).

Reglas de negocio: la comparación se hace por **checksum**, no por fecha de modificación — la fecha puede ser inconsistente entre sistemas de archivos y zonas horarias; el checksum es la única fuente confiable de "¿es realmente el mismo contenido?".

3.3 SincronizarArchivo / DescargarCambiosRemotos

Descripción: Envía (`SincronizarArchivo`) o recibe (`DescargarCambiosRemotos`) el contenido delta de un archivo específico ya clasificado como cambio sin conflicto. **Protocolo:** gRPC streaming (para soportar archivos grandes por partes).

Entrada (`SincronizarArchivo`): `token`, `sesionId`, `rutaArchivo`, `contenido` (stream de bytes), `checksumEsperado`. **Salida:** confirmación con `nuevaVersion` (delegado de Storage vía `SubirArchivo`).

Entrada (`DescargarCambiosRemotos`): `token`, `sesionId`, `rutaArchivo`. **Salida:** stream de bytes + metadatos de versión (delegado de Storage vía `DescargarArchivo`).

Errores: `STG-003` (checksum no coincide, propagado desde Storage), `SYNC-006` (la versión remota cambió durante la transferencia — requiere reiniciar la comparación para ese archivo específico).

Regla de negocio: este servicio **no reimplementa el versionado**; delega en Storage y solo agrega la capa de detección de cambios y orquestación de la sesión. Storage sigue siendo la única fuente de verdad de versiones.

3.4 CompararArchivos

Descripción: Compara explícitamente dos versiones (local vs. remota, o dos remotas) de un archivo para determinar si son idénticas, y en caso contrario producir el detalle de diferencia (útil para que el cliente decida cómo resolver un conflicto). **Entrada:** `token`, `archivoId`, `checksumA`, `checksumB` (o `versionA`, `versionB`). **Salida:** `sonIdenticos` (boolean), `detalleDiferencia` (metadatos: tamaño, fecha, autor de cada versión — no el contenido completo, por eficiencia). **Error:** `STG-102` (alguna versión referenciada no existe).

3.5 ResolverConflicto

Nombre: `ResolverConflicto` **Descripción:** Aplica la decisión de resolución sobre un archivo marcado como `CONFLICTO` por `DetectarCambiosLocales`. **Protocolo:** gRPC

Entrada:

Nombre	Tipo	Obligatorio	Descripción
`token`	string	Sí	—
`sesionId`	UUID	Sí	Sesión donde se detectó el conflicto
`archivoId`	UUID	Sí	Archivo en conflicto
`estrategia`	enum	Sí	`MANTENER_LOCAL`, `MANTENER_REMOTO`, `MANTENER_AMBOS` (crea una copia con sufijo), `FUSION_MANUAL` (el cliente ya resolvió y envía el contenido final)
`contenidoFusionado`	binary	Solo si `estrategia=FUSION_MANUAL`	Contenido final decidido por el usuario

Salida: `versionResultante`, `confirmacion`.

Errores: `SYNC-007` (conflicto ya resuelto por otra sesión concurrente — condición de carrera entre dos dispositivos resolviendo el mismo conflicto), `SYNC-008` (estrategia `FUSION_MANUAL` sin contenido provisto).

Reglas de negocio:

- La política de resolución de conflictos por defecto del sistema, si el usuario no interviene manualmente en un tiempo configurable, es **`MANTENER_AMBOS`** — nunca se descarta contenido de forma silenciosa sin que el usuario lo decida explícitamente. Esta es la decisión de diseño más difícil de cambiar después (afecta directamente la confianza del usuario en el sistema) y debe quedar fijada en la Fase de Diseño, no improvisada durante el desarrollo.
- `MANTENER_AMBOS` genera una copia con sufijo (ej. `archivo (conflicto - dispositivo X - fecha).ext`), delegando en Storage la creación de esa nueva entrada.

Seguridad: Requiere token válido y ser parte de la sesión de sincronización (`sesionId` debe pertenecer al usuario del token).

Dependencias: Distributed Storage Service (para persistir la resolución).

3.6 ProgramarSincronizacion / ConfigurarPoliticaSincronizacion

Entrada (ProgramarSincronizacion): `token`, `dispositivoId`, `rutaBase`, `frecuencia` (cron-like o intervalo simple). **Salida:** `programacionId`, confirmación.

Entrada (ConfigurarPoliticaSincronizacion): `token`, `estrategiaConflictoPorDefecto`, `excluirPatrones` (array de patrones de archivo a ignorar, ej. `*.tmp`). **Salida:** confirmación.

Regla de negocio: la política configurada es por usuario, no por dispositivo — dos dispositivos del mismo usuario deben resolver conflictos de forma consistente entre sí.

3.7 ObtenerEstadoSincronizacion / ObtenerHistorialSincronizacion / CancelarSincronizacion

Patrón estándar de consulta/cancelación. **Errores relevantes:** `SYNC-009` (sesión no encontrada o ya finalizada, para `CancelarSincronizacion`). **Regla de negocio:** cancelar una sincronización en curso debe dejar el estado parcial ya sincronizado como válido (los archivos que ya se transfirieron no se revierten), y marcar la sesión como `CANCELADA` para que el resto de cambios pendientes se retomen en la siguiente sincronización.

3.8 RegistrarDispositivo

Entrada: `token`, `nombreDispositivo`, `sistemaOperativo`, `identificadorHardware` (para generar `dispositivoId` estable). **Salida:** `dispositivoId`. **Error:** `SYNC-010` (dispositivo ya

registrado para ese usuario — se devuelve el `dispositivoId` existente en lugar de duplicar).

---

4. Modelo de datos

SesionSincronizacion

Atributo	Tipo	Descripción
`id`	UUID	—
`usuarioId`	UUID	—
`dispositivoId`	string	—
`rutaBase`	string	—
`estado`	enum	`EN_PROGRESO`, `COMPLETADA`, `CON_CONFLICTOS`, `FALLIDA`, `CANCELADA`
`fechaInicio` / `fechaFin`	datetime	—

CambioDetectado

Atributo	Tipo	Descripción
`archivoId`	UUID	—
`tipo`	enum	`NUEVO_LOCAL`, `NUEVO_REMOTO`, `MODIFICADO_LOCAL`, `MODIFICADO_REMOTO`, `CONFLICTO`, `ELIMINADO_LOCAL`, `ELIMINADO_REMOTO`
`checksumLocal` / `checksumRemoto`	string	—

ConflictoVersion

Atributo	Tipo	Descripción
`archivoId`	UUID	—
`versionLocal` / `versionRemota`	int	—
`estrategiaAplicada`	enum (nullable)	Se llena al resolverse

PoliticaSincronizacion

Atributo	Tipo	Descripción
`usuarioId`	UUID	—
`estrategiaConflictoPorDefecto`	enum	`MANTENER_AMBOS` por defecto del sistema
`excluirPatrones`	array<string>	—

Dispositivo

Atributo	Tipo	Descripción
`id`	string	—
`usuarioId`	UUID	—
`nombreDispositivo`	string	—
`sistemaOperativo`	enum	`WINDOWS`, `LINUX`, `ANDROID`, `IOS`
`ultimaConexion`	datetime	—

5. Eventos que genera el servicio

Evento	Disparado cuando	Consumidores típicos
`SincronizacionIniciada`	`IniciarSincronizacion`	Monitoring Service
`SincronizacionCompletada`	Sesión finaliza sin conflictos pendientes	Cliente (notificación), Monitoring
`SincronizacionConConflictos`	Se detectan conflictos que requieren intervención	Cliente (notificación destacada)
`ConflictoResuelto`	`ResolverConflicto` exitoso	Monitoring Service
`DispositivoRegistrado`	`RegistrarDispositivo`	Monitoring Service (inventario de dispositivos activos)

6. Integración

- **Quién consume:** clientes de escritorio principalmente; Web y móviles de forma limitada.
- **Quién responde:** File Sync Service, delegando en Storage toda la persistencia real.
- **Protocolo:** gRPC.
- **Momento del flujo:** posterior a Authentication (para identidad) y en constante interacción con Storage durante toda la sesión de sincronización (no es un flujo de un solo request-response, es una conversación de varios pasos).

---

7. Consideraciones de escalabilidad

- **Qué puede crecer:** `DetectarCambiosLocales` con Homes muy grandes (miles de archivos) puede volverse costoso si se compara archivo por archivo de forma secuencial — conviene procesar en lotes o usar estructuras tipo árbol de hashes (similar a un Merkle tree) para detectar subárboles sin cambios sin inspeccionar cada archivo individualmente.
- **Cuellos de botella posibles:** muchos dispositivos sincronizando simultáneamente contra el mismo Storage puede saturar el ancho de banda disponible — conviene throttling por usuario/dispositivo.
- **Mejoras futuras:** sincronización delta a nivel de bloque (no de archivo completo) para archivos grandes que cambian parcialmente; priorización de sincronización por tipo de archivo.

---

8. Casos límite

Operación	Caso límite	Comportamiento esperado
`IniciarSincronizacion`	El mismo usuario inicia sincronización desde dos dispositivos al mismo tiempo, misma ruta	Ambas sesiones proceden, pero si generan cambios sobre el mismo archivo, el segundo en llegar a Storage lo detecta como conflicto en su siguiente ciclo
`SincronizarArchivo`	Se pierde la conexión a mitad de la subida	La subida parcial no se confirma como versión válida (delegado del comportamiento de Storage); el cliente debe reintentar en la siguiente sesión
`ResolverConflicto`	Dos dispositivos intentan resolver el mismo conflicto con estrategias distintas casi simultáneamente	Debe aplicarse una resolución optimista con verificación: el segundo intento detecta que el conflicto ya no existe (`SYNC-007`) y se le informa al usuario en lugar de sobrescribir silenciosamente
`DetectarCambiosLocales`	Un archivo fue eliminado localmente y modificado remotamente al mismo tiempo	Se clasifica como `CONFLICTO` (no como eliminación automática) — eliminar nunca "gana" automáticamente sobre una modificación
`ProgramarSincronizacion`	Se programa una frecuencia menor al tiempo que tarda en completarse la sincronización anterior	La siguiente ejecución programada se posterga hasta que la actual termine, nunca se ejecutan en paralelo sobre la misma ruta

Contrato listo para implementación independiente de File Sync Service. Siguiente en la secuencia: HPC Service + Scheduler.